

```

-sha256

# create a KEK certificate
$: openssl req -new -x509 -newkey rsa:2048 -subj "/CN=My
Key Exchange Key/" -keyout KEK.key -out KEK.crt -days 36500
-nodes -sha256

# create a db certificate
$: openssl req -new -x509 -newkey rsa:2048 -subj "/CN=My
Signature Database/" -keyout db.key -out db.crt -days 36500
-nodes -sha256

```

These are all standard x.509 certificates with a 2048-bit modulus RSA key, valid for 100 years. They should last.

We'll now obtain the Microsoft KEK and db certificates. The public keys are available online. If you can't find them, you can simply extract them from your firmware. The two Microsoft db keys—named the *Microsoft Windows Production PCA 2011* and the *Microsoft Corporation UEFI CA 2011*—are available at <http://bit.ly/1elGfK6>. Simply copy the two code listings to two new files, naming them *db-ms.crt* and *db-ms-tp.crt*

Obtaining the existing KEKs is a bit hard. Your systems manufacturer will include at least two KEKs in the system—the *Microsoft Corporation KEK CA 2011* certificate, which is Microsoft's certificate, and a vendor certificate, which is for your systems manufacturer. They should be in your system; however, they're not necessary to boot Windows, Linux, the BIOS Setup Utility or update your firmware—which are all that you should care about. Additionally, if these KEKs are not in your system, you can be sure no one is enrolling or revoking db keys behind your back.

Now that we have our PK and KEK certificates and keys, we can start doing some signing. Let's start by converting the certificates to EFI Signature Lists, or ESLs, as follows:

```

$: for x in PK KEK db db-ms db-ms-tp;
> do
> cert-to-efi-sig-list -g $(uuidgen) $x.crt $x.esl;
> done;

```

You should now end up with quite a few EFI Signature List files. ESLs are made so that multiple signatures can simply be concatenated into a single file. We'll need to concatenate all the dbs together. So let's use the following command:

```

$: cat db.esl db-ms.esl db-ms-tp.esl > db-all.esl

```

We'll now need to sign these files. Let's sign the PK using the PK itself, then the KEK using the PK, and the db using the KEK:

```

$: sign-efi-sig-list -k PK.key -c PK.crt PK PK.esl PK.auth

```

```

$: sign-efi-sig-list -k PK.key -c PK.crt KEK KEK.esl KEK.auth
$: sign-efi-sig-list -k PK.key -c PK.crt db db-all.esl db-all.
auth

```

Now comes the really boring part — signing everything we want to boot with the db keys. At a bare minimum, you're going to have to sign your kernel binary and the UEFI shell. To sign a particular file, issue the following command:

```

$: sbsign -k db.key -c db.crt -o somefile.signed somefile

```

...replacing the term *somefile* with the actual file you want to sign. For example, to sign the kernel (which resides at */boot/vmlinuz-linux-ck* for me), I'd have to type the following command:

```

$: sudo sbsign -k db.key -c db.crt -o /boot/vmlinuz-linux-ck.
signed /boot/vmlinuz-linux-ck

```

And then do some renaming, like shown below:

```

$: sudo mv /boot/vmlinuz-linux-ck /boot/vmlinuz-linux-ck.
unsigned
$: sudo mv /boot/vmlinuz-linux-ck.signed /boot/vmlinuz-linux-
ck

```

...because the firmware will only look for the file */boot/vmlinuz-linux-ck*.

Now that the certificates have been generated and the binaries have been signed, let's enrol all of this into our firmware.

Flipping the switch

This is where things begin to get a bit foggy. I won't be able to give you exact instructions on how to do things here, because they vary from firmware to firmware. I'll only be able to tell you your objective—you'll have to find the exact option in the firmware to achieve your objective yourself. My machine has an awesome UEFI interface, and HP laptops are so bad they won't let you switch to *Setup Mode* from the firmware. Your mileage will definitely vary, but if you've got a Phoenix UEFI (ThinkPads and Acer Aspires have them, among others), your life should be easier.

Begin by downloading a disk image and *dd-ing* to a USB drive, as follows:

```

$: wget -c http://blog.hansenpartnership.com/wp-uploads/2013/
sb-usb.img
$: sudo dd if=sb-usb.img of=/dev/sdb bs=4M

```

Once that's done, copy the three *.auth* files we generated to the USB drive. Now reboot into the firmware. Once you're in the firmware, enable *Secure Boot*, and look for an option